GPIE Development Log

GREEN POLICY INTELLIGENCE ENGINE (GPIE)

Running log of every decision, data source, bug, and pivot in this project. Format: one entry per work session, most recent at the bottom.

GPIE is a geospatial framework for independently checking whether an environmental policy actually produced a measurable environmental effect, rather than trusting government-reported outcomes. Instead of relying on administrative statistics, it pulls satellite-derived indicators (NO_2, NDVI, climate, land cover, elevation, population) alongside GDP and policy-text data, and runs proper causal inference against them. The European Green Deal — specifically the European Climate Law's entry into force on 30 June 2021 — is the demonstration case: does EU-27 tropospheric NO_2 actually drop by more than a comparable set of non-EU European countries after that date? The architecture is built to be reused for any country or policy, not just this one.

Index

Entry 1

Before any satellite data could mean anything, I needed the "policy" half of the comparison — an actual structured record of what the European Green Deal consists of, not a vague reference to it. EUR-Lex is the official EU legal repository, so I built a scraper against it rather than collecting anything by hand, starting from a single test request with `requests` and BeautifulSoup and working up to something that could walk every search result automatically.

The build went in stages: first just getting a title and link out of one page, then iterating across a full results page to

pull title, URL, and legal status for every record, then following each link to the individual policy page to pull the actual document text. From there I layered on metadata extraction — policy type and year parsed straight out of the document heading, word count and character length computed directly, the CELEX identifier parsed out of the URL, a short summary pulled from the first line of the heading, and a simple keyword-based tagger to flag climate-relevant policies. Everything got written out to both `documents.json` and `documents.csv`, and I ran a first pass of exploratory analysis on it in Pandas — frequency counts by policy type, average length by year, a couple of bar charts — mostly as a sanity check that the dataset was complete and internally consistent before spending real time on the much heavier satellite pipeline.

Once the scraping worked end to end, I went back and refactored it properly: pulled metadata extraction, tagging, status parsing, and export into their own functions (`extract_metadata()`, `generate_tags()`, `export_json()`, `export_csv()`, `load_dataframe()`), wrapped the whole thing in a single `scrape_policies()` call, and routed everything through a `main()` entry point instead of a sequential script. Same output, same numbers — just something I could actually build on instead of a script I'd be afraid to touch. That mattered because the same architecture pattern (modular, fault-tolerant, reusable) was about to get reused for the satellite pipeline, and I wanted that pattern proven on the easier dataset first.

Entry 2

With the policy side done, I locked down the acquisition methodology before touching any actual satellite data — study area (EU), temporal window (Jan 2019–Dec 2024, giving a real pre-Green-Deal baseline plus several years of implementation), CRS (WGS84 throughout, no reprojection unless a future analysis specifically needs one), and the full list of datasets the framework would need: Sentinel-5P NO_2, Sentinel-2 NDVI, ESA WorldCover, Copernicus DEM, ERA5 climate, WorldPop population, Eurostat GDP, and NUTS administrative boundaries.

For NO_2 specifically I locked the protocol in detail: RPRO (reprocessed) Level-2 products over NRTI/OFFL, a 0.05° processing grid, the ESA-recommended `qa_value ≥ 0.75` quality threshold, NaN for missing pixels rather than any interpolation, and a month-wise download→process→cleanup lifecycle so raw orbital files don't pile up on disk. None of this was guesswork — each choice traces back to an actual reason (RPRO for long-term trend analysis rather than real-time monitoring, bounding-box discovery because polygon queries time out, monthly batching to match the pipeline's fault-isolation granularity), and I wrote it down explicitly so I wouldn't second-guess it mid-build later.

Then I got authentication actually working against the Copernicus Data Space Ecosystem — OAuth2 password grant, a reusable token function, and a first successful authenticated request against the product catalogue, confirming I could retrieve real Sentinel-5P product metadata (ID, name, dates, footprint) before building any download logic on top of it. Filtering the catalogue down to just the NO_2 products intersecting the European bounding box and the study window worked cleanly on the first real attempt.

Entry 3

Built out the actual download and config layer: a centralized `config.py` holding every study parameter (bounding box, dates, API endpoints, directory paths) so nothing was hardcoded twice, a download utility with file-existence and size-verification checks so re-runs skip anything already downloaded correctly, and automatic directory creation so the pipeline doesn't fail on a missing folder. The whole month-wise lifecycle — discover, differential-download, preprocess, verify, clear raw files — locked in as the standard shape every dataset in this project would follow.

Got HARP (the scientific processing library for Level-2→Level-3 conversion) working in the project's conda environment after sorting out the dependency chain, then validated it against one real Sentinel-5P product: confirmed the variable inventory, pulled out `tropospheric_NO2_column_number_density` with its lat/lon coordinates using HARP's `keep()` operation, and exported to CSV to check the extraction hadn't silently dropped or corrupted anything. It hadn't — full spatial coverage preserved, raw floating-point values intact, no filtering applied yet. That single-product validation became the template the full batch pipeline would run at scale.

Entry 4

Went through the pipeline hardening a lot of projects skip until something breaks. First, security: credentials had been sitting in plaintext in `auth.py`, which is fine for a quick test but not for a repo I intend to actually publish — moved everything into a `.env` file via `python-dotenv` and expanded `.gitignore` to keep secrets, cache files, and raw `.nc` data out of version control. Cleaned up `get_access_token()` to return a plain string instead of a full response object, which had been quietly causing `AttributeError`s downstream.

Then the download layer: rebuilt it so one failed file doesn't take down the whole batch — three retries with backoff per file, post-download size verification against the API's own `ContentLength` metadata, and a live test that confirmed both the skip-already-verified behavior and the retry behavior under a manual interruption. Wrapped the HARP preprocessing step into a proper `preprocess_file()` function with the quality filter and spatial binning baked into the same operation chain, returning `None` on failure instead of raising, so a bad file doesn't crash the whole run.

Built out the temporal side too — `generate_monthly_ranges()` in `date_utils.py` to produce clean month boundaries for any year range, and `run_pipeline.py` rebuilt as a real month-wise orchestrator with structured logging to a `logs/` directory, fault isolation at both the month level (a failed download logs and moves on) and the file level (a failed preprocess keeps the raw file around for a retry rather than deleting it). Tested the whole cycle on a single month before trusting it against the full 72-month run.

Entry 5

With the NO_2 pipeline hardened, I picked up three more datasets in the same session. Copernicus DEM (30m elevation) turned out to be a public, no-auth S3 bucket — wrote the tile-name generator matching the official naming convention, added remote size verification via HTTP HEAD requests before downloading anything, and confirmed a single live tile (Netherlands, 10.2 MB) landed correctly before committing to the full ~1,500–3,000 tile check across the whole European extent.

NUTS administrative boundaries came from Eurostat's GISCO API as a single GeoJSON — much simpler, no per-tile logic needed. The raw file included every NUTS entity though (EFTA members, candidates, non-members), so I built an explicit EU-27 ISO2/ISO3 mapping to filter it down to exactly 27 countries, including handling Eurostat's non-standard "EL" code for Greece. That filtered list became the reusable country roster every other per-country dataset in the project would drive off.

WorldPop population turned out to be the one dataset I couldn't fully close out this session. The REST API's verified "Global 1" dataset only covers 2000–2020, so I scoped execution explicitly to 2019–2020 rather than quietly leaving a gap or guessing at an unverified access path for 2021–2024 (which lives in WorldPop's newer "Global 2" dataset via HDX, a different system I hadn't tested yet). Wired it to iterate over the same EU-27 list from the NUTS work, with FTP-to-HTTPS URL normalization since WorldPop's file references are legacy FTP. Left 2021–2024 as an explicitly open item rather than papering over it — by the end of the day the pipeline stood at: NO_2 hardened but not yet run at full scale, DEM verified at one tile, NUTS complete, Population partial by design, and the causal-inference and economic modules not yet started.

Entry 6

Tried to get NDVI moving and hit a real dead end. Raw Sentinel-2 tiles are too large for full EU/6-year coverage to be practical on local storage, and the obvious cloud alternative (Sentinel Hub's API) needed a separate OAuth setup I hadn't built yet — so I went looking for a pre-computed NDVI product instead, since NDVI is just `(NIR - Red)/(NIR + Red)`, a standardized formula with no real loss of methodological control versus computing it myself.

Landed on Copernicus Global Land Service NDVI (300m, 10-daily, Version 3) via the CLMS portal. Registered an account, generated a service-account key (a JWT-based credential, structurally different from every other auth pattern in this project so far), and started building against CLMS's documented M2M download API. Also found — and deliberately avoided — an alternate route through the same dataset via CDSE's newer openEO API, since that pathway fails silently on a malformed request rather than erroring loudly, which is exactly the kind of thing you don't want to discover mid-batch. Left this as a documented next step rather than something to force through in a rush: register, get the token, read the actual M2M docs in full, test on one file, then scale.

Entry 7

Eurostat regional GDP (`nama_10r_2gdp`) turned out to be the simplest acquisition in the whole project — a single unauthenticated REST call returns every NUTS region in one response, no per-country looping needed, no retry logic even worth adding given how low-risk a small JSON response from a stable government API actually is. Verified the exact dataset code against multiple independent sources rather than trusting memory, filtered to the study period server-side via the API's own date parameters, and wrote a straightforward differential-download check so re-running the script doesn't refetch a file that's already there.

Entry 8

ESA WorldCover (10m global land cover, 11 classes) lives on a public S3 bucket, same pattern as DEM. Picked version v200 (2021) over v100 (2020) since it's the more accurate, more current classification — and noted for later that comparing the two versions directly would conflate real land-cover change with algorithmic differences between the two model versions, so this project uses v200 alone as its baseline rather than attempting a v100-vs-v200 change analysis. Built the tile-name generator for WorldCover's native 3° grid (different from DEM's 1° grid) and modeled the acquisition script closely on the DEM one. Left the actual single-tile verification test as the next concrete step before committing to a full download.

Entry 9

ERA5 climate (2m temperature, precipitation) meant switching to an entirely different access pattern — the CDS API operates on an asynchronous job queue (submit, wait, download) rather than a direct GET request, though the client library handles that lifecycle internally so I didn't need to write any polling logic myself. Had to register a fresh ECMWF account since the whole Climate Data Store had migrated infrastructure and old credentials wouldn't carry over, and had to manually accept the dataset's licence via the web UI since that step can't be automated.

Lost a chunk of time to a genuinely silly config problem: authentication kept failing with a missing-file error even after I'd put the `.cdsapirc` file in what I thought was the right place. Turned out the file had first been saved in the wrong user folder, and after moving it, was still failing because it had silently picked up a hidden `.txt` extension from the text editor's default save behavior — something File Explorer wasn't showing me. A straight `dir` listing in the terminal caught it; renaming it fixed authentication immediately. Once that was sorted, `download_era5_year()` pulled all 12 months of a year in a single request, kept the spatial extent locked to the same European bounding box as everything else, and a live 2019 test came back clean before I trusted the full six-year run.

Entry 10

Turned the 233 raw WorldCover tiles into actual usable statistics, and it took three failed approaches to get there. GDAL wouldn't install cleanly via pip on Windows (needs compiled binaries pip can't build), so I went through conda-forge instead, then had to work around VS Code's terminal not picking up conda activation by just calling the environment's Python directly by full path.

Mosaicking the tiles into one logical VRT worked fine — a lightweight 128KB index file, no pixel duplication. Clipping it to the EU boundary at full 10m resolution didn't: GDAL wanted ~1.75TB of disk space for that, and the script didn't even catch its own failure properly (it printed success with no output file, because I hadn't checked `gdal.Warp()`'s actual return value). Dropped that approach entirely — country-level percentages don't need pixel-perfect continental rasters. Zonal statistics straight off the VRT at native resolution hit a corrupted tile (one file with genuinely incomplete pixel data, since WorldCover's download script doesn't do byte-level verification the way DEM's does) and then an out-of-memory error even after fixing that. Resampling to 100m still ran out of memory on at least one large country.

What actually worked: resample to 500m using nearest-neighbor — deliberately not averaging, since WorldCover values are categorical class codes, and interpolating between "cropland" and "built-up" produces a meaningless number, not a real class — then run `rasterstats.zonal_stats()` against the NUTS boundaries. That gave clean per-country land-cover percentages across all 11 classes, saved to `landcover_stats_by_country.json`. Same lesson as the storage failure: match the processing resolution to what the analysis actually needs, not to the source data's native resolution.

Entry 11

Processed the six raw ERA5 yearly files and immediately hit a format surprise: `xarray` couldn't open them, and a quick check with `zipfile.is_zipfile()` confirmed they were actually ZIP archives wearing a `.nc` extension — a known quirk of the newer CDS infrastructure. Worse, each ZIP unpacked into two separate files, one for temperature and one for precipitation, split by an internal GRIB field-type distinction I hadn't anticipated. Wrote `unzip_era5.py` to handle both layers automatically across all six years rather than hardcoding one.

Merging the two variables per year hit a metadata conflict (`expver`, an internal CDS bookkeeping field with differing values between the two files) — dropped that coordinate before merging and converted units at the same time (Kelvin→Celsius, meters→millimeters, since neither of ERA5's native units is something a policy reader would find

interpretable). Then aggregated the gridded data down to per-country monthly statistics via `rasterio`'s geometry masking against the NUTS boundaries, skipping the handful of very small territories whose area doesn't intersect any grid cell at ERA5's coarse ~31km resolution — an expected limitation, not a bug. Ended up with 5,472 country-month records across all six years.

Entry 12

GDP was the easy one to process — no geospatial handling, no resampling, no memory pressure, just decoding Eurostat's JSON-stat format. Read up on the actual spec rather than guessing: dimensions are stored as ordered category indices, and values live in a flat dictionary keyed by an encoded integer that has to be decoded back into per-dimension indices via successive modulo/floor-division. Wrote that decode logic by hand rather than pulling in an external JSON-stat library, mostly so the whole thing stays auditable rather than a black box. Flattened the result into 18,470 records (spanning every NUTS level, not just country-level, since the source dataset mixes them) and wrote straight to CSV with the built-in `csv` module — no pandas needed for something this simple.

Entry 13

NDVI took two real attempts to land. The first — CGLS's direct M2M API, the route I'd deliberately chosen earlier specifically to avoid Sentinel Hub's OAuth complexity — worked through authentication cleanly (a four-step JWT-bearer flow, confirmed working once requests started returning data-validation errors instead of auth errors) but then failed on every single download attempt. Fixed the bounding-box format, fixed a server-side area-size limit by chunking the request into a 15×15 spatial grid, tried NUTS-country-code restriction instead of bounding boxes — none of it mattered, because the actual blocker was architectural: this specific dataset isn't physically hosted on CLMS at all, it's a reference into a Sentinel Hub "Bring Your Own Collection" entry. No amount of correctly-formatted CLMS requests was ever going to work.

So I rebuilt on Sentinel Hub directly, using the `byoc_collection` ID I'd found while diagnosing the CLMS dead end. Set up a fresh OAuth Client, wired authentication through the same Client Credentials flow, and used the Statistical API rather than raw raster downloads, since it computes zonal statistics server-side and returns them straight as JSON — collapsing acquisition and processing into one step for this dataset. Hit one evalscript bug early (`dataMask` needed a matching output declaration), then noticed the first successful values were in the 0–250 range, not physically meaningful NDVI (-1 to +1) — a digital-number encoding, not raw NDVI. Read the actual CGLS product manual rather than guessing at a conversion formula, confirmed `real NDVI = DN × 0.004 − 0.08`, and moved that conversion server-side into the evalscript itself.

Ran the full batch — 27 countries × 6 years — and 26 came back clean on the first try. France failed every year with a resolution-limit error, traced to its NUTS geometry including overseas territories (French Guiana, Réunion, Martinique) that blow out the effective bounding area. Clipped France's geometry to the same European bounding box everything else already uses, re-ran it alone, and it passed. Full batch: 162 records, all 27 countries, all six years.

Entry 14

Used a quiet stretch (waiting on downloads for NO_2, DEM, and Population) to actually test whether the four datasets I'd already processed — Climate, Land Cover, GDP, NDVI — could merge together at all, rather than assuming they would and finding out during the real analysis. Good thing I checked: Climate and Land Cover both included non-EU entities (their zonal-stats scripts had iterated over the full 39-country NUTS file rather than the EU-27 list I'd already built elsewhere), and GDP mixed multiple NUTS levels and — as it turned out — multiple measurement units in the same unfiltered column.

Built a shared `filter_eu27.py` utility so every dataset needing EU-27 filtering pulls from one place instead of re-deriving the list, and filtered Climate, Land Cover, and GDP down, keeping a `_full` backup of each original in case broader scope is useful later. GDP's filter produced 1,134 records instead of the expected 162, which led me into the same country-year having seven different GDP values ranging from 123 to over 3.5 million — multiple measurement units (per-capita euro, million euro, million national currency, several PPS variants) that the original decoding logic had silently dropped from the output. Fixed the decoder to keep the unit label, picked `MIO_EUR` (absolute GDP at current prices) as the project standard, and re-filtered correctly to 162. Along the way I made the mistake of restoring GDP from an old backup that predated the unit fix, which just reintroduced the bug — the actual fix was re-running the decoder from source, not restoring anything. Final scope check across all four datasets came back clean and consistent.

Entry 15

Applied the same Sentinel Hub Statistical API pattern to NO_2 that had just worked for NDVI, since the original RPRO/HARP raw-acquisition pipeline — fully built and verified at small scale — turned out to be impractical at the real 27-country × 6-year scale (roughly 55–60 orbital files a month × 72 months, with skip-checking alone eating several minutes per run). Rather than force that through, I moved to server-side aggregation the same way NDVI had.

Hit one evalscript error early — NO_2's quality filtering isn't a per-pixel band the way I'd assumed, it's a request-level `processing.minQa` parameter, and Sentinel Hub's own documented default for that value is 75, which happens to line up exactly with the ESA-recommended threshold I'd already locked for this project. No compromise needed there. Reused the same bounding-box geometry clipping built for France's NDVI fix, and the full 162-request batch (27 countries × 6 years) came back with zero failures on the first attempt — no France-style anomaly this time, since the fix was already baked in before I ran it. The original RPRO pipeline stays in the repo, untouched and functional at small scale, in case a future need calls for raw Level-2 access again.

Entry 16

Ran the EU-27 consistency filter against GDP and NO_2 and cleared out two small bugs along the way — `filter_gdp()` was still trying to filter on a `unit` column that no longer existed, since an earlier session had already fixed the unit-mixing problem at the source. Removed the now-obsolete filter condition and drop line, and GDP filtered clean at 162→162. NO_2 also came back 162→162, confirming the acquisition itself had already been EU-27-scoped from the start — the filter step here was a formal confirmation rather than an actual correction.

Also made a real decision about Population: only 2019–2020 has ever downloaded successfully, and completing 2021–2024 would mean integrating a different WorldPop data source I hadn't tested. Since every other dataset spans the full six years, I decided Population won't go into the core causal model as a control variable — a variable with four years of missing data would do more harm than good there. It stays as a supporting, descriptive dataset instead. DEM's filter is still blocked on its download finishing, and NO_2's raw nested structure still needs flattening before it can join the master merge — both carried forward.

Entry 17

Flattened NO_2 from its raw nested Sentinel Hub structure into a flat per-country-year-month table, and the first run came out 162 records short of the expected 1,944 — exactly the number of country-year combinations, which was too clean a number to be a coincidence. Grouped the output by country-year and found all 162 combinations had exactly 11 months, not 12. A second check by month number confirmed it precisely: December was missing everywhere, every single country-year, zero exceptions.

Traced it to the acquisition script's time-range parameters — both ended at `{year}-12-31T23:59:59Z`, one second short of a complete monthly interval under Sentinel Hub's half-open `[start, end)` window logic. A real December interval needs to end at `{year+1}-01-01T00:00:00Z`. Fixed both time-range blocks, re-ran the full 162-request acquisition, and re-flattened — this time producing the full 1,944 records with all twelve months present everywhere. The uniformity of the original gap (100% of records, exactly one month) was the actual diagnostic clue here — a random or partial gap would have pointed somewhere else entirely.

Entry 18

Closed out the last few processing gaps and assembled the real master dataset, and found three separate bugs doing it.

DEM processed cleanly on the first attempt — 860 tiles into one VRT, resampled to 500m with bilinear interpolation this time rather than nearest-neighbor, since elevation is continuous and averaging between values is scientifically valid here, unlike land cover's categorical codes. Filtered from 39 to 27 records without issue.

NDVI turned out to have the exact same December gap NO_2 had just been fixed for — same Sentinel Hub half-open-interval issue, independently present because NDVI and NO_2 are acquired by separate scripts. Applied the identical fix, re-ran, and it came back clean.

The master merge itself surfaced three bugs. First, land cover values had landed in the merged table as a literal stringified dictionary instead of individual numeric columns — fixed by detecting the nested structure and flattening

each class into its own column. Second, a `ValueError` on CSV writing because different countries have different sets of land-cover classes present, so deriving fieldnames from just the first row broke on any later row with an extra class — fixed by collecting the union of all keys across every row before writing. Third, and the one that took the longest to track down: `avg_temp_c` was null in literally 100% of rows while `avg_precip_mm`, pulled from the same lookup dictionary in the same code path, was fully populated — a logical impossibility that ruled out a merge-logic bug outright. Traced it to the raw ERA5 file actually containing 3,888 records instead of the expected 1,944, because temperature and precipitation source files carried slightly different internal timestamps for the same calendar month, and combining them without explicit time alignment caused xarray to treat those as genuinely separate time steps — two half-populated rows per month instead of one complete one. Forced the precipitation dataset's time coordinate onto the temperature dataset's before combining, re-ran the full ERA5 pipeline, and the final master dataset — 1,944 rows, 21 columns — came out with zero missing values anywhere except the legitimate satellite-retrieval gaps in NO_2 and NDVI themselves.

Entry 19

Built the actual causal model, and it took a real research-design detour plus a genuinely nasty environment failure to get a trustworthy result out of it.

Difference-in-Differences was the obvious first choice, but GPIE's actual setting doesn't fit it cleanly — the European Green Deal applies to all 27 EU countries simultaneously, so there's no natural control group inside the dataset. Checked whether policy-intensity variation across countries could substitute for one; it couldn't, since the EU regulations in the policy database apply uniformly with no country-specific tagging. Redesigned around a timing-based approach instead: the European Climate Law's 30 June 2021 effective date as a single treatment point, with country fixed effects and seasonal controls doing the rest of the identification work.

The first implementation (via `linearmodels`) ran without error but produced no output at all — a silent failure. Turned out entity fixed effects already fully absorb any time-invariant variable like elevation or land cover, so including them explicitly alongside entity effects created perfect collinearity. Removed them as explicit regressors. The silent failure persisted anyway, for a more fundamental reason: with full time fixed effects and a treatment variable that's identical across all 27 countries for any given month, `treatment` is mathematically indistinguishable from the time fixed effects themselves. Swapped full time effects for coarser calendar-month dummies instead, which let the treatment variable's before/after variation actually be estimable — at the documented cost of no longer controlling for one-off time shocks like COVID-era disruption.

Even after both fixes, the model kept dying silently with no Python exception at all. Chased this one hard: the exit code pointed to a Windows-level crash below the interpreter, not a Python error. Swapped `linearmodels` for `statsmodels` — same crash. Stripped out clustered standard errors — same crash. Forced single-threaded execution in case it was an Intel MKL threading issue — same crash. Rewrote the whole thing to bypass the formula API and build the design matrix by hand — same crash. Reduced it all the way down to `numpy.linalg.lstsq()` on a random matrix of the same shape — still crashed. Tried an alternate solver, then isolated it to a bare matrix multiplication with no solving step at all — still crashed. That finally nailed it: a broken Intel MKL backend in the conda environment, not a single line of project code. Reinstalling numpy/scipy with the `nomkl` package (forcing OpenBLAS instead of MKL) fixed it immediately.

With the environment actually working, the model ran clean: treatment coefficient -2.285×10^{-6}, p=0.026, 95% CI entirely negative, R^2=0.39, n=1,748. A statistically significant drop in NO_2 following the Climate Law — the first real result out of this project's core question.

Entry 20

Pushed the model harder before trusting it. Ran the identical specification against NDVI as a second outcome — small negative coefficient, p=0.128, not significant, which read as plausible rather than disappointing: the Climate Law's direct instruments (emissions trading, industrial regulation) connect to NO_2 much more immediately than to vegetation health, which responds to land-use policy on a longer timescale.

Then ran a placebo test on the NO_2 result — same model, but with the treatment date artificially shifted to 30 June 2020, a date with no comparable policy event. It came back *more* significant than the real result (p=0.002 vs. 0.026), which is a clean placebo failure: a model that finds a "significant effect" at an arbitrary date can't be trusted on the real one either. Added an explicit linear time trend as a diagnostic, and with it included, the real-date coefficient dropped to p=0.408 — no longer significant. That confirmed exactly what the placebo test implied: the original result was picking up a general multi-year NO_2 decline across the whole study period, not anything specific to this one law.

This isn't a coding bug or a data problem — it's a structural identification limitation. With no untreated comparison group, a single-cohort, time-only treatment design mathematically can't separate "the law caused this" from "NO_2 was already falling for unrelated reasons and kept falling through the treatment date." The fix is a real control group, not a better model spec. Picked the UK, Norway, and Switzerland — geographically and economically comparable non-EU European countries, none subject to the Green Deal — and scoped out what that would require: new boundary data (GADM, since NUTS is EU-specific), extended acquisition of NO_2/NDVI/climate for three more countries, and a second GDP source since Eurostat doesn't cover non-EU countries.

Entry 21

Built the actual control group. Boundaries for the UK, Norway, and Switzerland came from GADM at country-outline level, verified for correct codes, valid geometry, and matching CRS before I trusted them. Built `country_boundaries.py` as a single shared interface that loads geometry from either NUTS (EU-27) or GADM (control group) through the same function, standardizing both to a consistent two-letter code convention and reusing the bounding-box-clipping fix already proven on France.

Extending NDVI to all 30 countries surfaced two new failures: France again (the clipping wasn't yet enabled by default in this script) and a brand-new one for the UK — a `COMMON_BAD_PAYLOAD` error traced to the sheer geometric complexity of its unclipped GADM boundary (~2,562 polygon parts across all its small islands). Enabling bbox clipping fixed France immediately; the UK needed a second fix, since clipping its already-complex geometry against a bounding box produced a mixed `GeometryCollection` (degenerate points and lines mixed in with real polygon area) that Sentinel Hub's API rejects outright. Extended the clipping logic to filter down to just the polygon components, and the UK went through clean. NO_2 extension to 30 countries hit zero failures on the first attempt, since both fixes were already in place before I ran it.

Climate needed a different kind of fix — Norway and Switzerland were already present in the NUTS file as EFTA members, so pulling them again from GADM double-counted them, producing exactly double the expected record count for those two countries specifically. Fixed by explicitly skipping any NUTS entry matching a control-group code. GDP for the control group came from the World Bank API instead of Eurostat (which doesn't cover non-EU countries), with an explicit, documented EUR/USD conversion using annual average rates — an acknowledged approximation, acceptable for a control variable rather than a primary one.

With all four datasets extended and a new `treatment_group` column added, the real two-group DiD model ran clean: interaction coefficient -1.40×10^{-6}, p=0.663, CI spanning zero. Once measured against an actual non-EU comparison group, there's no statistically distinguishable EU-specific NO_2 reduction — whatever decline happened within the EU-27 looks like it's part of a broader trend shared with comparable non-EU countries, not something specific to this one piece of legislation. Not the result I might have hoped to find at the outset, but the one the validation process actually supports.

Entry 22

Ran one more robustness layer on the control-group result: an event study, splitting the single before/after comparison into 23 separate quarterly coefficients (2019Q1–2024Q4, relative to 2021Q2) rather than one averaged effect. This answers two questions a single average can't: did the EU-27 and control group already look different before treatment (which would break the model's core assumption), and could a real but delayed effect have gotten averaged away across the whole post-period?

All 23 quarters came back non-significant, p-values scattered between 0.18 and 0.94 with no discernible pattern before or after the treatment quarter. That's two pieces of good news at once — no pre-treatment divergence (supporting the parallel-trends assumption the DiD design leans on) and no hidden delayed effect either (so the null result isn't just an artifact of averaging). Combined with the placebo test and the control-group correction, that's a three-part validation sequence backing the same conclusion from three different angles, which is a meaningfully stronger basis than any one of them alone. Module 8's core causal-inference work is done at this point: no statistically distinguishable EU-specific NO_2 effect, once genuinely checked against a real comparison group and stress-tested three separate ways.

Entry 23

Went to visualize the event-study coefficients and immediately lost time to an environment problem that had nothing to do with the plotting code itself. Matplotlib wasn't installed yet; installing it via conda completed without

error, but the very next script run failed on a `DLL load failed` error tracing into `_ctypes` — a core Python module, not even matplotlib itself. Something about resolving matplotlib's dependency tree had corrupted the interpreter installation in that environment. Force-reinstalling Python in place didn't fix it. Rather than keep debugging an environment of unknown extent of damage, I just built a fresh one from scratch with the full accumulated dependency list specified explicitly up front — including `nomkl` this time, given the MKL crash from a few sessions back.

With that sorted, the actual plot came together cleanly: 23 quarterly coefficients as points with confidence-interval error bars, a zero reference line, and a vertical line marking the treatment date. Verified it by having it opened directly and walked through against a checklist, since I couldn't view the image inline in this session — all 23 points present and correctly ordered, every interval spanning zero, matching the regression output exactly.

One more thing came out of actually looking at that plot: a non-significant result can mean either a genuine null effect or a study that's simply underpowered to detect a real one, and those have different implications for how honestly the finding should be reported. The overall DiD confidence interval — $[-7.12\times10^{-6}, +4.32\times10^{-6}]$ — is wide relative to the estimate, which is consistent with either explanation, and a three-country control group is a genuinely small comparison set for a panel model with this many fixed effects. Decided the documentation needed to say that directly rather than just reporting "no effect detected" — the honest version is "no statistically distinguishable effect at this sample size and control-group size," which is a meaningfully different, more careful claim.

Entry 24

Built out the full set of geospatial visualizations, using a Python-only pipeline (geopandas + matplotlib) instead of QGIS, since every input was already in a directly-usable format and I wanted the maps reproducible and version-controlled alongside the rest of the code. Verified each one by opening it locally and checking it against a specific checklist for what it was supposed to show, since I couldn't view images inline mid-session.

The NO_2 choropleth needed a colormap swap partway through — the initial yellow-to-red scale made several low-NO_2 countries visually indistinguishable from blank map background, fixed by switching to `plasma`, whose low end stays visible. The land-cover map crashed outright on a `nan` RGBA error, traced to the loading function assuming a flatter JSON structure than the actual nested one — fixed the extraction path and added a grey fallback for any future unmatched class. GDP's map had a real communication problem, not a bug: GDP is heavily right-skewed (a handful of huge economies dwarfing everyone else), and a linear color scale compressed nearly every country into one pale band. Fixed with a log transform plus a switch to `viridis`, spreading the color scale across the actual distribution instead of just the top few outliers, with the colorbar explicitly labeled as log-scale so it can't be misread as raw GDP.

Ended up with seven visualizations total — the NO_2 choropleth, a 2019-vs-2024 before/after comparison (deliberately sharing one color scale across both panels so any visible shift represents a real change, not a scaling artifact), a categorical study-design map explaining the treatment/control split, dominant land-cover class, the NDVI choropleth, the GDP choropleth, and the event-study plot from the prior session — stylistically consistent across the set and ready to go into the dashboard.

Entry 25

Built the dashboard and got the project onto GitHub for the first time. Streamlit's multi-page routing crashed immediately on launch because it scans the `pages/` folder and expects every referenced file to actually exist — created placeholder pages for everything planned before building content into them properly.

Styling went through a real rewrite, not just a tweak — the first pass used a pastel color scheme, which I rejected outright in favor of a dark, more technical aesthetic: near-black gradient background, cyan/purple/green accent gradient, glassmorphism cards, monospace numeric accents. Centralized all of it into a shared `styles.py` so every page pulls from one source rather than duplicating CSS. Caught one more structural bug — an early page had its own `set_page_config()` call duplicating the one in `app.py`, which is only valid once per session — removed it everywhere except the home page.

Populated eight pages end to end: home, environmental data, study design, before/after, economic context, causal results, methodology & limitations, and about/data with a downloadable master dataset. Then went back and added real interactivity rather than leaving it as a static image gallery — a Plotly-based, country-selectable time-series explorer with control-group countries rendered in a distinct dotted line style, plus a regression table and comparative bar chart on the Causal Results page.

Getting the repo onto GitHub took a few genuine first-timer bumps: Git wasn't installed, and even after installing it,

VS Code's terminal had already cached the old PATH at launch and needed a full app restart, not just a terminal restart, to pick up the new binary. `git add .` then failed with a dubious-ownership error (a D: drive filesystem thing, fixed via `safe.directory`), and the first commit failed on missing author identity, fixed with the standard one-time `user.name` / `user.email` config. Once past those, the push went through clean — about 134 files and 1.85 million lines, mostly the accumulated JSON datasets and generated images, onto a fresh public repository.

Entry 26

Deployed to Streamlit Community Cloud and every page except home broke immediately with a media-file error. Traced it to `PROJECT_ROOT` being computed one directory level short — from a page file two folders deep, the path math only walked up to `dashboard/`, not the actual project root, which local execution had somehow never exposed but Streamlit Cloud's different mount structure surfaced right away. Fixed the path calculation across all eight page files and converted the remaining relative path references to proper `os.path.join()` construction.

Cleaned up documentation discoverability at the same time — added a documentation table right at the top of the README, and chased down a false alarm about the README not rendering on GitHub (it was rendering fine; the file listing above it was just being mistaken for its absence, which is standard GitHub behavior, not a bug).

Noticed two real datasets — DEM and ERA5 climate — had been fully acquired and processed as control variables but never actually visualized anywhere, and that the before/after comparison had only ever been built for NO_2, leaving NDVI without an equivalent. Built all three: a DEM elevation map using a terrain colormap, an ERA5 temperature map across all 30 countries, and an NDVI before/after panel structurally mirroring the existing NO_2 one. Both before/after maps had a colorbar overlap issue from relying on matplotlib's automatic layout — fixed by switching to explicit axis placement with dedicated vertical space reserved. Reordered the dashboard pages so Study Design comes before Environmental Data, since showing results before explaining what's being measured had the sequence backwards, and folded DEM and climate into an expanded Control Variables page alongside GDP and land cover.

Finally, tested whether the "globally transferable" claim in the project's own stated design was actually true rather than just asserted — built a standalone script reusing only the existing Sentinel Hub authentication and evalscript logic, pointed at a simple bounding box over India instead of any EU country, with zero changes to the validated EU-27 pipeline itself. All six years came back clean with physically realistic NO_2 values, confirming the acquisition architecture genuinely is portable, not just claimed to be.

Entry 27

Went through the full repository end to end ahead of finishing this project properly, applying the same statistical rigor the NO_2 result had already gone through to everything else rather than treating that one validation as covering the whole project.

Found and fixed a handful of concrete issues: dashboard PDF download buttons using relative paths that only work if the working directory happens to be the repo root (same bug pattern I'd already hit and fixed on other projects); `requirements.txt` listing only 4 of roughly 15 packages the code actually imports, which would have made the README's own "run locally" instructions fail; duplicate image files sitting in the outputs folder; a leftover unresolved Git merge conflict marker in one of the documents; and inconsistent dataset/visualization counts across the README, the project overview, and the dashboard's own homepage metric, all reconciled to the same numbers.

The bigger fix was statistical: every causal-inference script was using plain OLS standard errors, which understate uncertainty for panel data with repeated monthly observations per country — a well-documented issue. Clustered standard errors by country across all five models. The headline DiD result moved from p=0.632 to p=0.663 — the null got *more* solid, not less, so this was safe to apply. The event study picked up three nominally significant quarters under clustering that hadn't shown under classical SEs — close to the roughly one false positive you'd expect by chance at this sample size, with no consistent direction, and I reported that plainly instead of leaving it out.

Ran five more robustness checks against the corrected model — removing GDP entirely (result moved closer to zero, so GDP wasn't driving anything), a log-transformed outcome (confirmed the null isn't a functional-form artifact), treatment-date sensitivity, a baseline-pollution heterogeneity split, and a formal minimum-detectable-effect calculation, which quantified what I'd only been describing qualitatively before: at 80% power this design can detect an effect of roughly 28% of baseline NO_2, while the actual coefficient sits at about 4.4% of that baseline — a real, previously-implicit limitation now stated as a number.

While extending clustering to every model, I found that the NDVI analysis had never actually gone through the

control-group correction the NO_2 analysis had — it was still running the original single-cohort design the placebo test had already proven unreliable for exactly this kind of problem. Rebuilt it to match the corrected NO_2 design, and it produced a genuinely different result: where the uncorrected model found nothing (p=0.128), the corrected two-group model finds a real, statistically significant relative NDVI decline in the EU-27 versus the control group (coefficient −0.0210, p=0.012). Reported honestly as a secondary finding, not evidence the Climate Law harmed vegetation outright — land-use change, drought, and agricultural policy aren't controlled for here — but as a real reminder that validation applied only to a project's headline result can leave something sitting unnoticed in a secondary one.

Entry 28

Reviewed the finished project once more specifically against the kind of questions a scholarship review panel would actually raise — control-group size, spillover risk into the control countries, how the NDVI finding should be interpreted, country-level versus finer spatial resolution. Rather than trying to fix everything at once, I triaged each candidate improvement by real cost against real value, since six more projects still needed the same treatment.

What I actually did: added a paragraph making the UK's 2020 EU exit an explicit, stated part of the control-group justification rather than leaving it implicit; added a Future Work section naming Synthetic Control Method, finer spatial resolution, formal spatial-autocorrelation diagnostics, and a delayed-effect test as specific next steps, so an obvious "why didn't you try X" question gets a direct answer instead of silence; added an architecture diagram and reproducibility section to the README; and set up a `CITATION.cff` file for eventual publication. What I deliberately didn't do: actually build the synthetic control or spatial diagnostics themselves, since each is a real data and modeling effort in its own right, not a documentation fix — named as future work instead of quietly skipped. No new empirical claims came out of this pass, just presentation and documentation improvements sitting on top of already-validated analysis.

Entry 29

Went back through every quantitative claim in the paper and independently recomputed it from the master datasets directly, rather than trusting numbers that had accumulated across a lot of separate sessions. Everything checked out to the reported precision except one real inconsistency: the two earliest, already-superseded single-cohort models (the original NO_2 result and the original NDVI result) had been reported with classical, non-clustered standard errors, even though every later model in the project uses cluster-robust SEs by country — that standard just never got applied retroactively to those two once it was adopted.

Recomputed both properly. The NO_2 number moved from p=0.026 to p=0.041 — still significant, so it doesn't change that section's conclusion. The NDVI number moved from p=0.128 ("no effect") to p=0.0017 — genuinely significant, which changes the accurate story: the original single-cohort NDVI model was already picking up a real effect under this project's own stated methodology, not just after the control-group correction. The corrected two-group model (p=0.012) is still the trustworthy, reported result — its role is better identification, isolating an EU-specific effect from a shared regional trend, not first-time detection.

Everything else — the placebo test, the time-trend diagnostic, the corrected two-group NO_2 and NDVI models, the event study, all five robustness checks, the minimum-detectable-effect number — matched to the reported precision with no changes needed. Spot-checked three of the paper's citations against independent searches and confirmed them real and correctly attributed; flagged the remaining ten as not individually re-verified this round rather than silently treating them as checked. Fixed the two inconsistent passages across the paper, the project overview, and the dashboard, and reframed the NDVI narrative to say what actually happened — no underlying data or model logic changed, this was purely a standard-error consistency fix.

Entry 30

Built two more checks against the corrected NO_2 model, both already named in Future Work rather than left as vague intentions.

Synthetic control first: with all three control countries as donors, Norway's NO_2 series turned out to be missing 29 of 72 months — a real acquisition gap, not something random — which collapsed the usable pre-treatment window down to 12 months if I kept it in. Dropped Norway from the donor pool entirely rather than imputing over a 40%-missing series; UK plus Switzerland alone keeps 28 of 30 pre-treatment months intact. Fit convex NNLS weights (UK 21%, Switzerland 79%) with a ridge-augmented bias correction on the residual. The post-treatment gap came out

the same sign and roughly the same size as the DiD coefficient. With only two donors, the in-space placebo test isn't a real permutation test — said so directly rather than dressing it up as more rigorous than it is.

Then spatial autocorrelation: built country geometries from the existing boundary files, used KNN-4 weights on projected centroids rather than contiguity, since several countries here are islands that would end up disconnected under simple adjacency. Global Moran's I on raw NO_2 levels came back strongly clustered (0.522, p=0.001), which is exactly what you'd expect physically — pollution doesn't respect borders. The actually useful check is on the DiD model's residuals, and that came back not significant (I=0.020, p=0.247) — meaning the country and month fixed effects are already absorbing the spatial dependence, and clustered standard errors aren't missing anything a spatial model would need to fix. Added Local Moran's I on top to see *where* the clustering sits, and found a high-pollution cluster across Benelux, Germany, and Denmark and a low-pollution cluster across the Nordics and Baltics.

Wrote both into the paper as new methodology and results sections, removed the two corresponding items from Future Work since they're now actually done, and updated the limitations section to note the synthetic control's donor pool is thinner still than the DiD's control group, and that Norway's coverage gap is a real, acknowledged data-quality issue.

Entry 31

Expanded the control group from 3 countries to 9 — added Iceland, Albania, Bosnia and Herzegovina, Montenegro, North Macedonia, and Serbia, plus a full clean re-fetch of Norway's NO_2 series to check whether its gap was ever fixable. All six new countries already had NUTS geometry in the existing boundary file, so no new boundary work was needed; climate came free from the ERA5 grids already downloaded, just a wider zonal-stats loop over the same files.

Norway's gap didn't fix — still 29 of 72 months missing after a completely clean re-fetch, which settles it as a real high-latitude satellite coverage limit, not the stale-file problem I'd assumed earlier. Iceland has the same issue at a smaller scale. Both stay excluded from the synthetic control donor pool, documented plainly rather than smoothed over.

Re-ran everything downstream that depends on the control group, not just the new acquisition — NDVI, the event study, all five robustness checks (consolidated into their own script this time, since they'd never had one), synthetic control, spatial autocorrelation, and every map or plot touching the control group, so nothing was left showing old 3-country numbers next to new 9-country ones.

The extra power actually changed the story, not just tightened the same null. The pooled DiD coefficient moved to -2.22×10^{-6}, p=0.101 — closer to conventional significance but still not there on its own. A heterogeneity split by baseline pollution level came back significant for higher-baseline countries specifically (mostly Western/Central Europe: p=0.003, coefficient -5.46×10^{-6}), something the 3-country group didn't have the power to detect (it had shown p=0.245 on the same split). The event study now shows four significant post-treatment quarters, all negative, all landing in Q2 or Q3 — a real seasonal pattern, not scattered noise the way the earlier 3-quarter result had been. Treatment-date sensitivity threw a genuine oddity: shifting the date six months earlier is itself significant (p=0.021) while the true date isn't (p=0.101) — flagged that honestly rather than picking whichever date looks better, since it might mean the effect's real onset doesn't line up exactly with the legal date, or might mean something else entirely. The log-transform check went the other direction from every other check here, shrinking toward zero — which actually makes sense given the effect looks concentrated in a subset of countries rather than a uniform percentage decline everywhere, and I noted that rather than treating it as a contradiction.

None of this overturns the pooled null as the headline number — the interaction term's confidence interval still crosses zero. But three independent things now point the same direction (the heterogeneity split, the event study, the date sensitivity), which the smaller control group simply didn't have the power to distinguish from noise. Rewrote the paper's headline numbers, abstract, and every affected section to describe the 9-country design directly rather than as an update from 3, regenerated all 11 affected maps and plots, and wrote this up as an honestly open, partially-resolved finding rather than forcing it into either "still null" or "actually significant."